



作 品 概 要

JobHunting平台



PART 01

背景

供需两端的多重困境

求职者困境：

海量职位信息真假难辨、更新滞后，导致求职者难以获取实时、可靠的行业与岗位动态，陷入“信息过载”泥潭，决策效率低下。

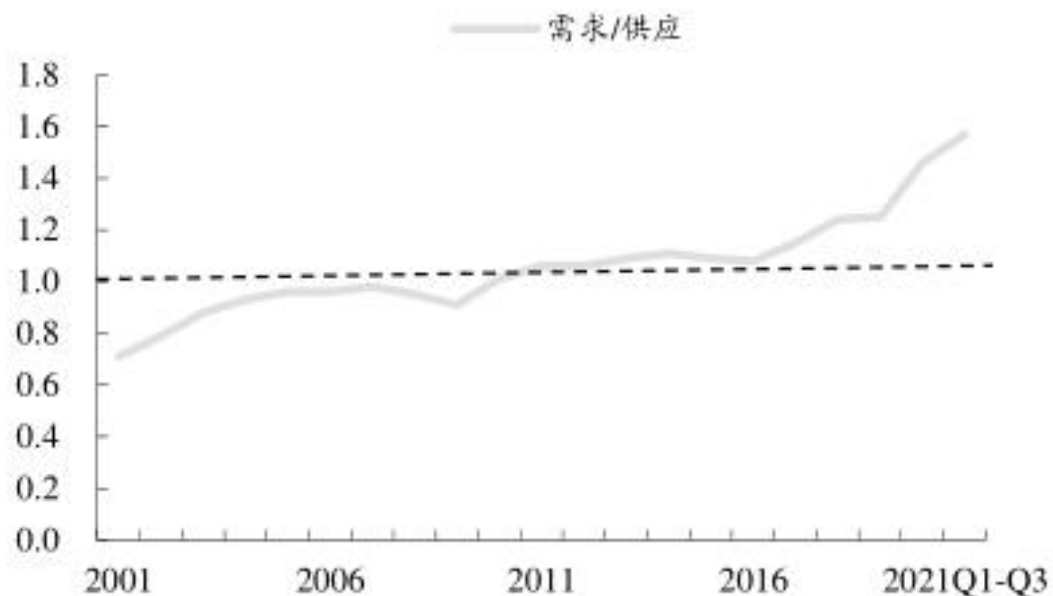
简历投递常因与岗位要求（JD）的显性或隐性需求匹配不足而石沉大海，个性化求职辅导资源匮乏。缺乏针对目标公司和岗位的有效准备信息，难以在激烈竞争中脱颖而出。

企业招聘困境：

从海量简历中筛选合格候选人耗时费力，人岗匹配精准度不足。

平台或渠道上的“信息失真”（如虚假职位、夸大描述）损害招聘方信誉，浪费双方资源。

图10：中国岗位空缺与求职人数比率



数据来源：人力资源和社会保障部，CIC，东吴证券研

究院

当前求职市场的信息困境深化分析

信息滞后性： 岗位需求变化快，但信息传递链条长，导致职位更新与实际需求之间存在“岗位更新延迟”，求职者错过黄金机会，企业也延迟填补关键空缺。

信息过载与噪声： 渠道分散、信息爆炸，求职者难以快速识别真正相关的优质机会，企业简历池充斥着大量低匹配度申请，造成“决策效率低下”。

信息失真与信任危机： “虚假职位泛滥”、描述夸大或模糊，严重损害求职者信任，增加筛选成本和风险，破坏市场健康度。

1. 传统招聘平台：数据失真与维度缺失

以智联招聘、前程无忧为代表的招聘平台，虽积累了大量岗位薪资信息，但其数据本质上是企业为吸引人才而发布的“广告价”，往往存在虚高或模糊区间。例如，某互联网公司为了争夺AI算法工程师，在招聘信息中标注“年薪30万-60万”，但实际入职薪资可能因候选人资历、企业预算等因素大幅波动。更关键的是，这类平台仅提供薪资范围，缺乏岗位职责、企业规模、地域差异等维度的交叉分析，难以支撑科学的薪酬体系设计或协助企业在招聘过程中定岗定薪。

2. 调研平台：滞后性与样本局限

传统调研机构通过线下问卷、企业访谈等方式收集数据，一份报告从立项到发布通常需要2-6个月。以某咨询公司2024年发布的《中国互联网行业薪酬白皮书》为例，其数据采集截止于2023年第三季度，但到报告发布时，AI大模型技术已引发新一轮人才争夺战，报告中的“市场平均水平”早已失去参考价值。此外，调研样本多集中于头部企业，中小型企业及新兴岗位数据覆盖不足，导致报告普适性存疑。

3. 薪酬数据平台：静态分析与场景割裂

部分企业采用薪酬数据平台进行内部管理，但这些工具往往局限于历史数据的静态统计，缺乏实时市场对标能力。例如，某制造业企业使用某平台进行年度调薪时，发现其提供的行业涨薪率数据与竞争对手实际动作严重脱节，最终导致核心技术人员批量流失。

RAG 赋能求职领域的变革潜力

RAG技术结合了强大的信息检索与智能内容生成能力，为解决上述困境提供了突破性的技术路径。

实时性破局： RAG能从动态更新、权威可信的数据源中实时检索最新职位、行业趋势和公司信息，彻底解决“信息滞后”问题。

精准匹配提效： 通过深度理解求职者画像和企业JD核心要求，RAG能进行精准语义匹配与个性化信息检索，大幅提升简历与岗位的适配度，减少无效投递，同时为企业智能筛选高潜候选人。

智能降噪与洞察： RAG能够智能整合、提炼、总结海量信息，为求职者生成简明扼要的职位要点分析、面试准备锦囊，消除信息噪音，提升决策效率。

可信信息增强： 通过对接权威数据源并利用智能验证逻辑，RAG可为用户显著过滤并警示风险信息，对抗“信息失真”，逐步重建求职者与招聘方之间的信任桥梁。

PART 02

原理

RAG 核心思想

用“检索+增强”弥补生成模型的先天缺陷

检索：

从权威知识库中实时查找与用户问题相关的文档片段。

增强：

将检索到的文档片段作为 证据上下文，插入到大模型的输入提示中。

生成：

大模型基于证据上下文 + 用户问题，生成最终回答。

工作流程：

大模型能够解析用户问题意图，提取关键词，并用Embedding模型将问题转化为向量。依据结果，在向量数据库中搜索相似度最高的文档片段，将检索到的文档片段作为“证据上下文”插入Prompt。最终，大模型基于“证据上下文”生成答案。

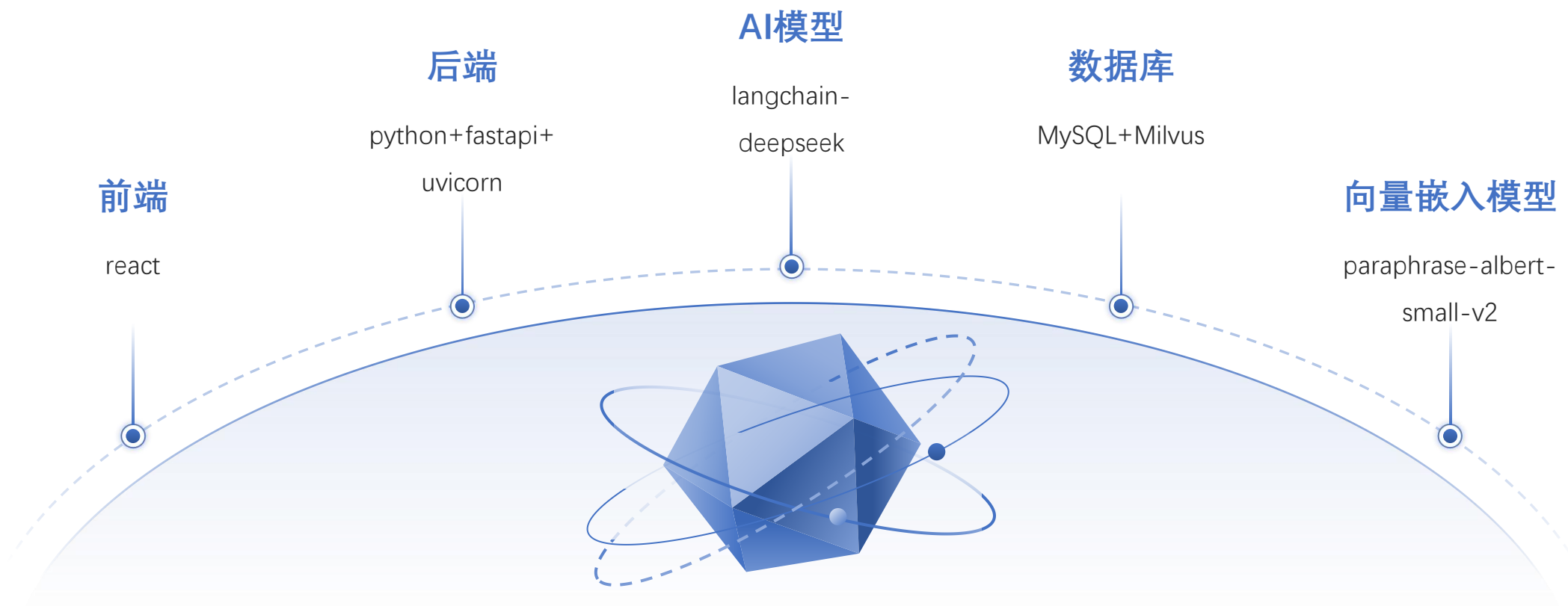
RAG vs 传统生成模型

能力	传统大模型	RAG
知识时效性	训练数据截止	实时更新（依赖外部库）
事实准确性	易产生幻觉	基于证据，可信度高
专业领域适应性	通用能力，细节弱	强垂直领域支持
可解释性	黑盒生成	提供数据来源（可溯源）

PART 03

核心技术支撑

主要涉及的技术



核心技术支撑

前后端分离架构

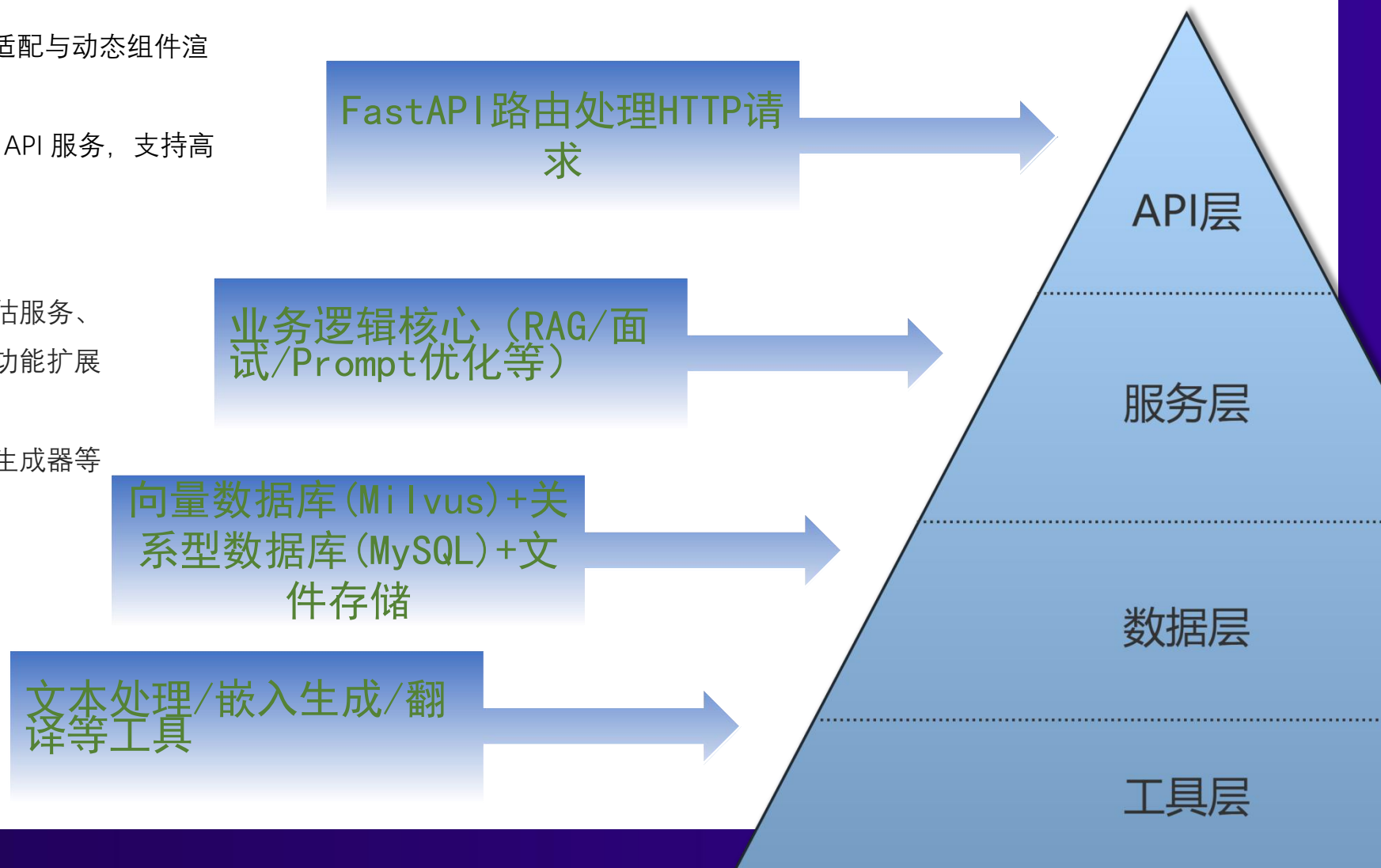
前端采用 React 构建响应式界面，支持多端适配与动态组件渲染，提升用户交互体验。

后端使用 FastAPI + Uvicorn 实现高性能异步 API 服务，支持高并发请求，响应速度快。

模块化服务设计

后端基于模块化设计，划分为检索组件、评估服务、任务管理器、提示管理器等功能模块，便于功能扩展与维护。

支持异步任务队列、可视化、评估器、提示生成器等多个子系统，满足多样化任务需求。



核心技术支撑

混合型数据库架构

关系型数据库 MySQL：管理结构化信息，如职位信息、面试细节等。

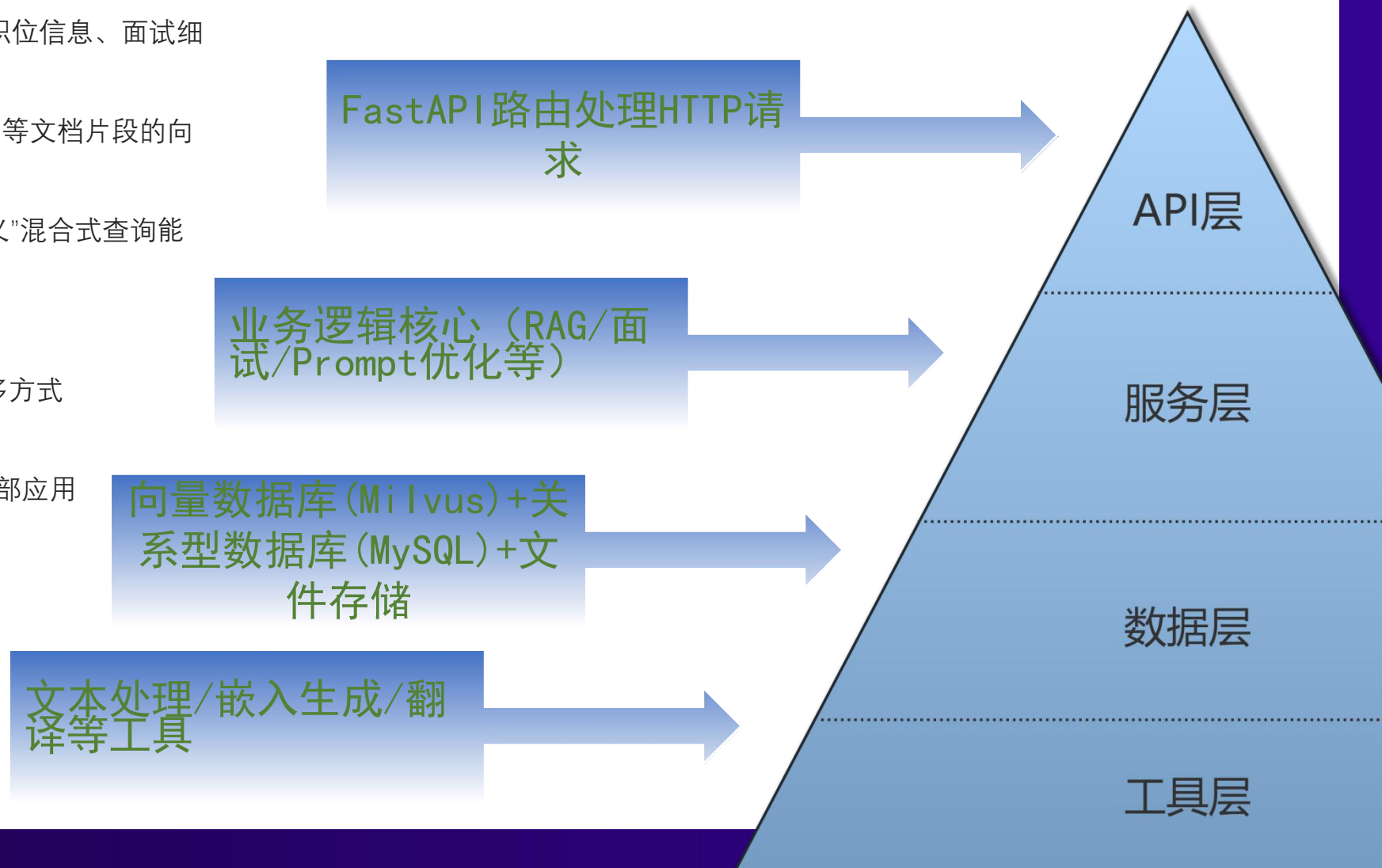
向量数据库 Milvus：存储 PDF、Word、JSON 等文档片段的向量表示，实现高效语义检索。

配合向量检索与结构化查询，形成“知识 + 语义”混合式查询能力。

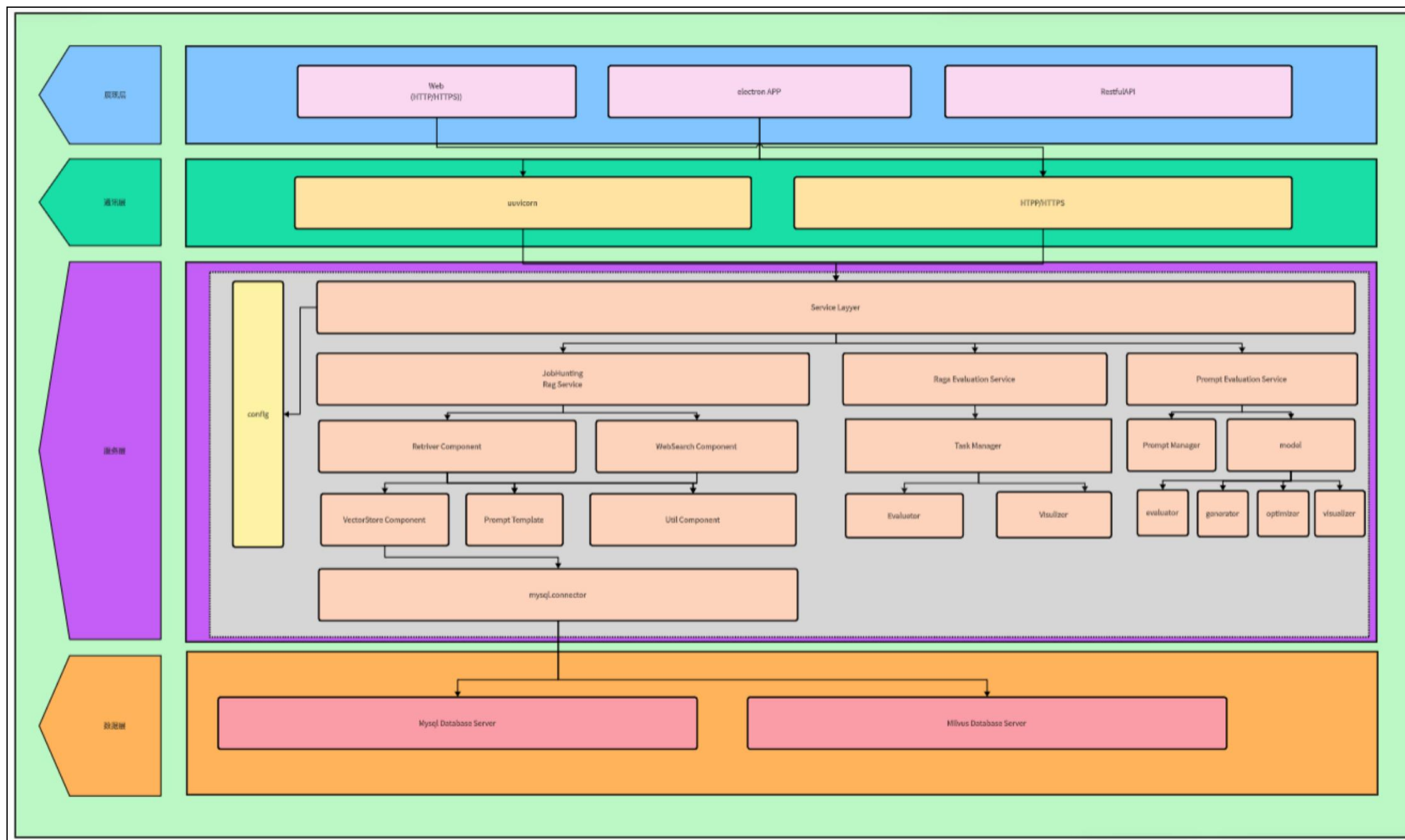
支持多端接入与API调用

系统支持 Web、Electron App 和 RESTful API 多方式接入，便于集成至各类平台。

接口通过 HTTP/HTTPS 暴露，可无缝对接内外部应用系统或服务组件。



系统总架构



数据库架构



PART 04

创 新 亮 点

创新亮点

1. 结构化 + 非结构化信息混合建模

将结构化字段（如公司、职位、薪资等）与文档型数据（如面试问题段落）统一向量化处理与检索，实现对职位信息的深度语义分析与匹配。

2. 端到端求职知识智能管理系统

提供从信息采集、问答训练、语义查询到评估优化的一站式服务链路

3. 可组合 Prompt 模板和评估机制

支持不同提示模板与 LLM 接入模型的自由组合与性能评估，支持提示工程自动调优与效果可视化展示。

4. 支持动态问答与文档匹配

利用 Milvus + FastAPI 框架，在海量面试资料中实现实时向量查询与相似度匹配，用于候选人快速定位答案、模拟问答等

5. 技术选型轻量高效，部署灵活

全栈采用轻量化、容器友好型技术栈，适合部署在本地开发机、云服务器或校园私有云环境中。



PART 05

应用价值

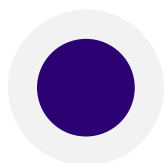
主要应用价值

该平台通过 “RAG+职业服务” 深度融合，将碎片化求职知识转化为结构化智能服务，在提升求职成功率（精准匹配）、降低决策成本（趋势导航）、加速能力成长（测评+训练）三个维度创造不可替代性，同时为企业提供可扩展的知识管理基础设施。



智能化求职

通过AI精准推荐岗位、模拟面试训练和简历智能优化，显著提升求职效率与成功率



结构化知识管理

实现多格式文档上传、自动切分向量化及混合引擎检索，构建高效可控的知识库体系



全周期职业发展

提供从性格能力测评、岗位推荐到简历优化、面试训练的一站式职业成长服务闭环



PART 06

团队核心成员

团队介绍

我们是一支专注于RAG系统与大模型Agent的Scrum团队

胡杨

Scrum Master



职责:

- 组织并管理每日站会、Sprint规划会议、Sprint回顾会议和Sprint评审会议
- 确保Scrum流程的顺利进行,帮助团队成员克服工作中的障碍
- 维护和更新Scrum仪表盘,确保团队按时交付
- 作为Scrum小队的协调者,推动团队文化和团队合作的形成

王宇翔

Product Owner



职责:

- 负责项目的进度管理,确保关键交付时间的达成
- 维护Product Backlog,确保每个任务的优先级和价值清晰
- 确保项目需求的实现与验收,监督任务完成情况并与Scrum团队沟通
- 与Stakeholders保持密切联系,获取并确认需求,调整Product Backlog

万鹏举

会议记录者



职责:

- 记录每次Scrum会议的要点,包括Sprint计划、每日站会、Sprint评审和回顾等
- 使用Markdown格式组织会议记录,确保其清晰、简洁、易于追溯
- 追踪会议中的决策与行动项,并在下一次会议时进行跟进
- 将会议记录及时分享给团队成员,以便参考和落实

团队介绍

通过RAG系统和大模型Agent，为求职者提供全生态的建议

黄海锋

测试样例书写者



职责：

- 根据项目需求和开发进度，编写并维护自动化测试样例
- 确保测试覆盖所有功能模块，并且测试用例符合项目需求
- 与开发人员合作，确保测试用例的准确性和有效性
- 在Sprint结束前执行测试，提供测试结果和反馈

黄扬昊

CI/CD负责人



职责：

- 负责项目的CI/CD管道的搭建、维护和优化
- 确保开发流程中的自动化构建、测试和部署流程顺利运行
- 持续监控CI/CD流程中的问题，优化效率，减少部署中的风险
- 与团队协作，确保代码的持续集成与交付符合标准，保证系统稳定性

范家磊

关键文档书写者



职责：

- 负责编写和维护项目中的关键文档，如需求文档、设计文档、技术文档等
- 确保文档准确、清晰，能够帮助团队成员理解项目要求及技术细节
- 与开发人员和Product Owner合作，确保文档内容与实际开发过程相符
- 定期更新文档，确保其在整个开发周期中保持最新状态

PART 07

指导老师介绍



Zhiwei Jiang 蒋智威

PhD, Assistant Professor (姑苏青年教授 & 特聘研究员)

State Key Laboratory for Novel Software Technology,
School of Intelligence Software and Engineering,
Nanjing University, China

Email: jzw AT nju.edu.cn

Office: Room 224, East of Nanyong Building, Suzhou Campus

About Me

I am currently a tenure-track assistant professor in the [School of Intelligence Software and Engineering, Nanjing University](#). I received my Ph.D. degree from Nanjing University in 2018 under the supervision of Prof. [Qing Gu](#), and received my B.Eng. degree from [Central South University](#) in 2011.

Research Interests

1. Natural Language Processing

Large Language Model

Representation Learning

Text Classification

Text Mining for Software Engineering

2. Language and Vision

Multimodal Understanding

Text-to-Image Diffusion Model

3. Machine Learning

Deep Learning

Graph-based Learning

Weakly-Supervised Learning

Ordinal Classification

1 作品阐释

1.1 作品缘起：问题与需求发现

在数字化招聘持续演进的当下，求职者与招聘方之间的信息不对称依然严重。具体体现在：

- 求职信息冗杂、更新不及时，用户难以获取个性化、可信的职位数据；
- 简历撰写缺乏专业指导，模板化严重，难以体现个体优势；
- 传统求职平台推荐逻辑简单，无法基于用户深层特质进行精准匹配；
- 求职者自我认知模糊，缺乏科学的职业测评与路径规划支持；
- 缺乏对企业的深度了解和背景核查机制，导致就业风险上升；
- 面试环节准备不足，难以系统性训练表达与应对能力；
- 当前RAG类平台多聚焦于问答、搜索等单一任务，缺乏多模块一体化解决方案。

因此，急需构建一个融合语义理解、智能推荐、行为引导、人格分析与趋势分析于一体的智能化求职平台。

1.2 痛点与需求分析

痛点	对应功能与解决方案
① 信息不对称，搜索匹配不精准	→ 语义搜索 + 岗位精确搜索 ：利用向量语义匹配，解决传统关键词搜索误差大问题
② 简历模板化、不突出个性	→ AI简历生成器 ：基于求职者背景自动生成多版本个性化简历
③ 职业定位模糊，发展路径不清晰	→ MBTI+大五人格测评系统 ：融合心理与能力分析，生成个性化职业发展路径图
④ 岗位推荐不智能，无法动态应变市场变化	→ 强化学习推荐引擎 ：综合个人偏好与实时市场趋势进行多目标优化推荐
⑤ 面试准备缺乏系统性	→ AI智能模拟面试系统 ：语音/文本双模态交互，模拟HR常见提问，生成反馈建议
⑥ 缺乏对企业信任机制	→ 企业背景调查模块 ：聚合工商信息、员工评论、薪资透明度等进行综合评估
⑦ 缺乏对系统性能的量化评估	→ RAG评估平台 ：支持对问答质量、召回精度、生成完整度等进行量化测试

1.3 平台架构与模块设计

1.3.1 1. 平台核心功能模块

- **语义搜索与趋势分析**：集成BERT+BiLSTM模型实现搜索意图识别与实体抽取，智能捕捉用户搜索过程中的关键信息。系统将把这些信息与知识库内容进行对比分析，提取当前求职市场的热点趋势，并以趋势热点榜单的形式直观呈现，同时标注相关搜索的热度数据。
- **问答系统**：RAG结构，结合本地招聘知识库 + LLM生成答案
- **智能简历生成**：用户只需上传个人基本信息、教育背景、工作经历、项目经验、技能证书等资料，系统将基于预设的多种简历模板，一键生成专业、美观的简历文档。
- **岗位智能推荐**：岗位智能推荐系统通过实时数据管道实现分钟级岗位信息同步，确保推荐内容的时效性。系统采用基于增量学习的推荐算法更新机制，持续优化推荐效果。推荐引擎综合考虑用户画像特征与岗位需求矩阵进行智能匹配，生成个性化推荐流，同时通过智能去重机制基于用户行为日志过滤重复曝光内容并控制推荐多样性，提升用户体验。
- **职业测评与决策支持**：职业测评与决策支持系统采用MBTI+大五人格模型的混合评估框架，从技术硬实力和软技能两个维度进行全面评估。系统根据测评结果自动生成包含竞争力指数等关键指标的个性化报告，并绘制清晰的职业发展路径图。推荐策略引擎采用基于强化学习的多目标优化算法，综合考虑薪资、成长性等因素，并实时融合市场供需指数，确保推荐结果的准确性和实用性。
- **智能模拟面试**：AI面试功能模拟真实面试场景，为用户提供沉浸式的面试体验。用户进入面试界面后，AI将作为面试官，根据岗位要求和用户简历信息，随机生成一系列面试问题，涵盖专业知识、沟通能力、团队协作、问题解决等多个维度。用户通过语音或文字回答问题，AI将实时对回答进行分析评估，从回答的完整性、逻辑性、准确性等方面给出评分，并在面试结束后生成详细的面试报告。
- **企业背景调查**：企业背景调查平台为用户提供全面的企业信息查询服务。用户只需输入企业名称，系统将通过多渠道数据整合，快速展示企业的详细信息，包括但不限于企业基本信息、企业规模、经营范围等，帮助用户深入了解企业的背景和潜在风险。同时附有信息来源文章，确保信息真实性，也便于用户通过文章进一步了解企业。
- **岗位搜索系统**：精准工作搜索功能通过多维度的筛选条件，帮助用户快速定位到目标岗位。用户可以根据公司名称、岗位类型、工作地点等关键信息进行组合搜索，系统将实时匹配并展示符合条件的岗位列表。同时，搜索结果页面提供岗位详情的预览功能，用户可以快速浏览岗位职责、任职要求、公司介绍等核心信息，点击岗位名称即可进入详细页面，查看完整的岗位描述、福利待遇、面试流程等内容。
- **知识库管理面板**：系统提供专业的知识库管理面板，采用双栏布局设计分别展示数据库与知识库资源。通过实时统计在线数据库实例数、注册用户总量和支持的自然语言类型数量等关键指标，管理员可以全面掌握系统运行状态。系统支持钻取式查看数据库存储结构、索引状态及内存占用情况，并可视化呈现向量数据库的维度分布与聚类特征。数据维护方面，提供直观的操作界面支持单条数据编辑，同时具备批量导入功能，可自动解析PDF/CSV/JSON格式文件，并通过内置数据清洗规则引擎确保数据质量。此外，系统还提供混合架构监控功能，清晰展示知识图谱中结构化数据源与向量化数据的跨库关联关系映射，以及存储资源分配视图。
- **RAG评估平台**：

1.3.2 2. 技术实现要点

- 基于 **LangChain + Vector Store** 构建的语义检索系统
- 使用 **混合评估策略（结构化+文本）** 实现个性化职业测评
- **FastAPI + React** 构建前后端系统，界面清晰、交互友好
- 支持 **语音识别+语义生成模型** 实现问答系统
- 向量数据库(Milvus)+关系型数据库(MySQL)+文件存储

1.4 创新意义与价值

1.4.1 1. 技术创新点

- 将RAG扩展至多模态求职任务集，不仅限于文档问答，而是服务于简历生成、岗位推荐、测评路径等多个环节；
- 自主设计强化学习驱动的职业推荐系统，可根据市场变化动态优化推荐结果；
- MBTI + 大五人格模型联合评估，兼顾心理性格与工作偏好，提升测评准确性与多维性；
- 内嵌RAG评估平台，实现从检索到生成效果的全链路可解释、可量化闭环优化；
- 多模态支持（文本 + 语音 + 图形），提升用户体验，适应不同使用偏好；

1.4.2 2. 应用价值与社会意义

- 有效缓解求职者信息获取与匹配效率低下的问题；
- 提供个性化职业决策建议，辅助青年群体明确方向、规避风险；
- 帮助招聘平台构建更智能、更可解释的推荐系统，增强用户粘性；
- 在教育、实习与就业服务平台中具备强推广潜力，可用于高校职业发展中心、人才市场等场景；
- 推动RAG技术向纵深方向落地，成为生成式AI在就业与人才领域的重要应用示范；

2 具体实现内容

2.1 raga搜索模块

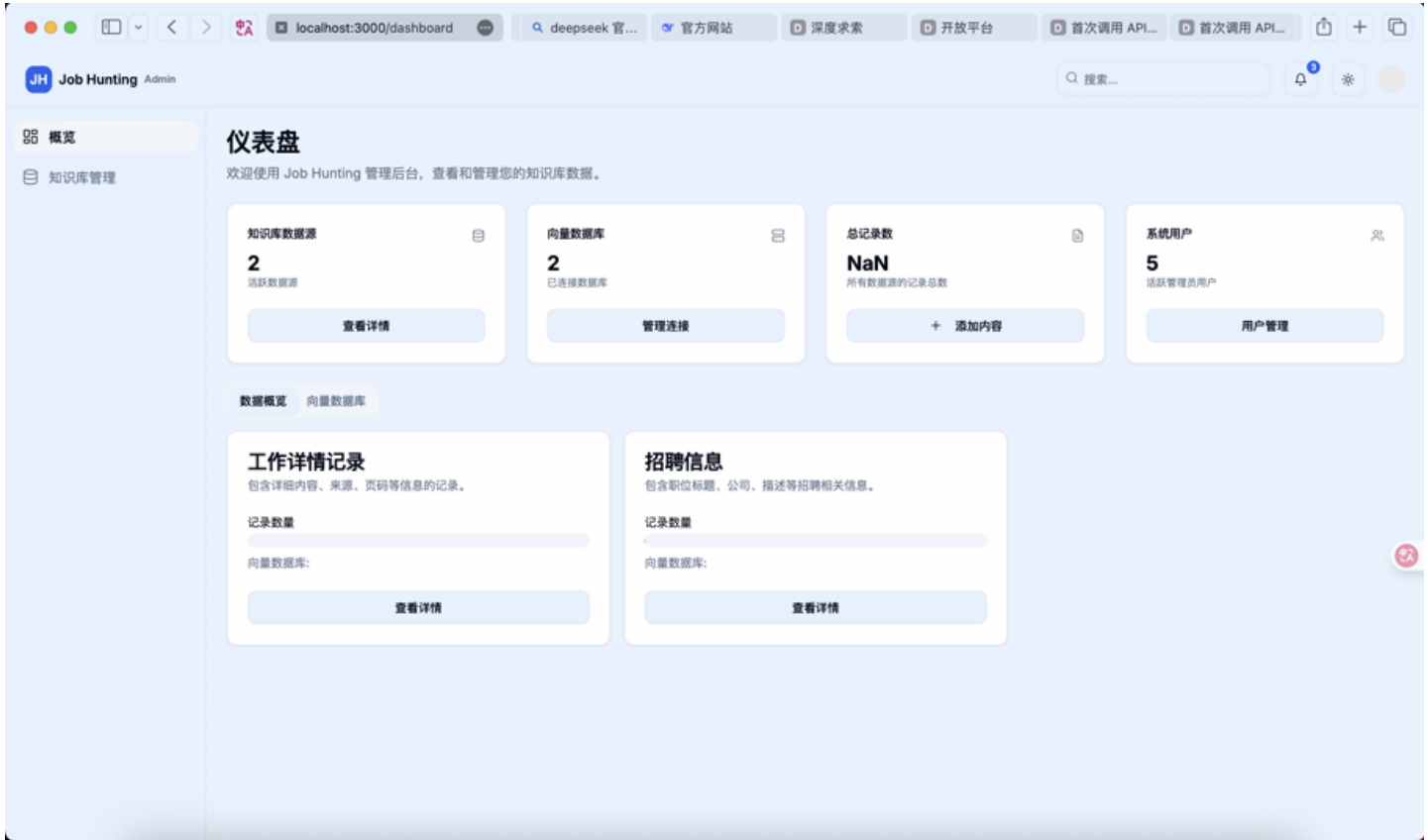
功能描述

RAG搜索模块是求职系统中的核心智能服务组件，融合了语义向量检索与大语言模型生成能力，结合我们搭建的向量数据库与web_search搜索到的网络信息，约束大模型输出，能够对用户输入的自然语言查询进行深度理解，检索与之相关的职位信息、公司介绍、行业知识、面试指南等多源内容，并生成自然语言形式的个性化回答。同时我们提供了上传文件，以及结合历史记录进行回答的特性，上传文件可以把pdf，word等文件上传给系统，系统自动识别有效信息，放入大模型进行回答，可结合用户简历等得到更符合用户要求的回答；结合历史记录可以使上下文回答更连贯，方便用户使用。

2.2 知识库管理面板

功能描述：

系统提供专业的知识库管理面板，采用双栏布局设计分别展示数据库与知识库资源。通过实时统计在线数据库实例数、注册用户总量和支持的自然语言类型数量等关键指标，管理员可以全面掌握系统运行状态。系统支持钻取式查看数据库存储结构、索引状态及内存占用情况，并可视化呈现向量数据库的维度分布与聚类特征。数据维护方面，提供直观的操作界面支持单条数据编辑，同时具备批量导入功能，可自动解析PDF/CSV/JSON格式文件，并通过内置数据清洗规则引擎确保数据质量。此外，系统还提供混合架构监控功能，清晰展示知识图谱中结构化数据源与向量化数据的跨库关联关系映射，以及存储资源分配视图。由于本部分功能较多，在开发过程中可以考虑使用一个单独的页面，要注意操作界面的简介性和易读性，让用户能够快速理解各组件的功能。同时对增改数据，批量导入等操作要注意性能问题。



2.3 语义搜索与趋势分析模块

功能描述：

语义搜索与趋势分析模块集成BERT+BiLSTM模型实现搜索意图识别与实体抽取，智能捕捉用户搜索过程中的关键信息。系统将这些信息与知识库内容进行对比分析，提取当前求职市场的热点趋势，并以趋势热点榜单的形式直观呈现，同时标注相关搜索的热度数据。通过该功能，能帮助用户能准确把握市场热点动向。对于自己的问题能有更清楚全面的认知，有助于其求职的过程。

2.4 语音搜索

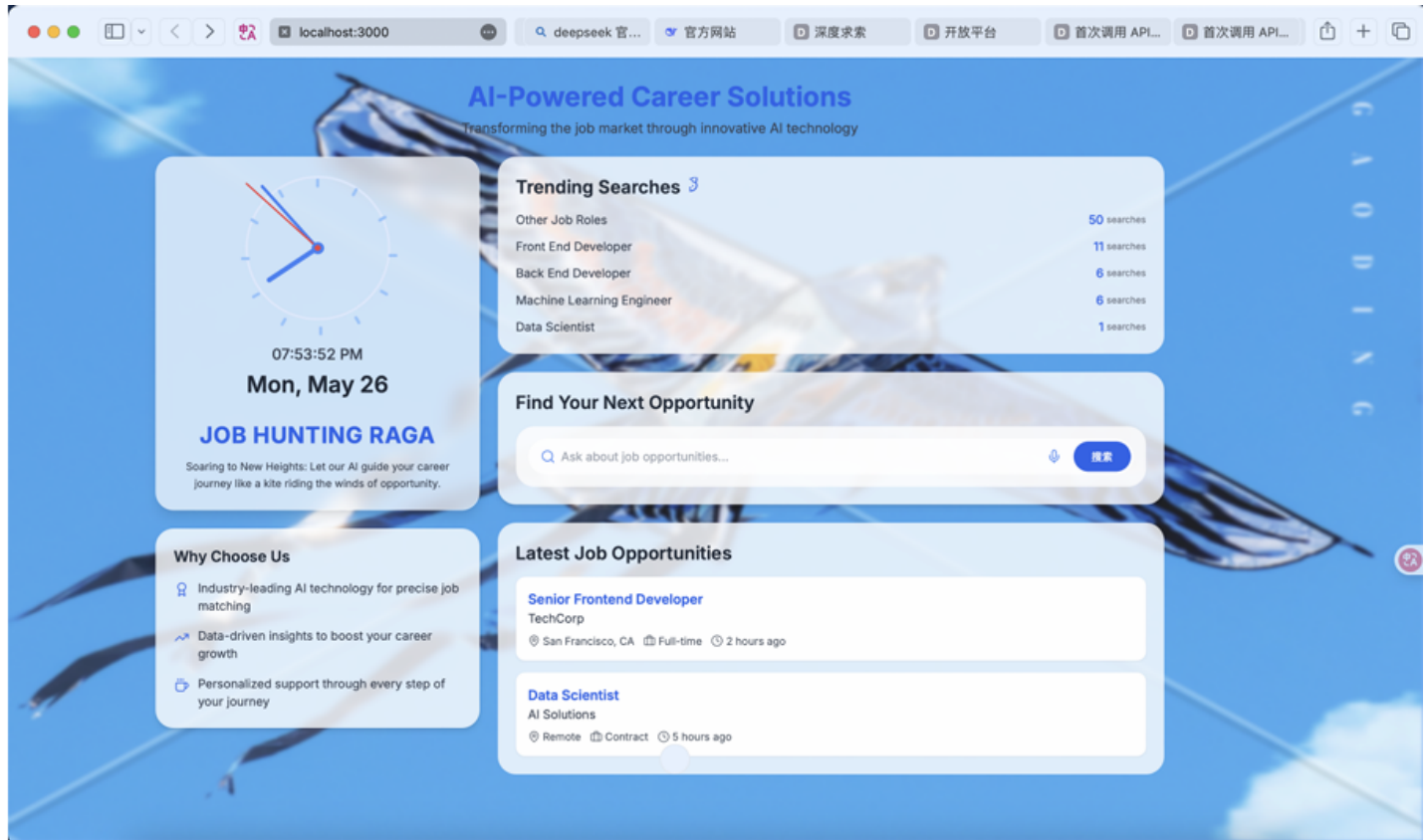
功能描述：

语音搜索功能为用户提供了更加便捷的搜索方式。用户在搜索栏点击语音输入按钮后，可以通过语音指令代替手动输入来搜索目标信息，如岗位名称、企业名称、关键词等。系统通过先进的语音识别技术，能够准确识别用户的语音指令，并快速返回搜索结果。语音搜索支持多种语言和方言，同时具备语音纠错功能，能够自动纠正用户的发音错误或模糊表达，确保搜索的准确性。此外，语音搜索还支持语音播报功能，用户可以选择收听搜索结果的语音摘要，提升搜索体验。开发时要注意语音识别的准确率和响应速度，让用户能够轻松、快速地完成搜索操作。

2.5 岗位智能推荐系统

功能描述：

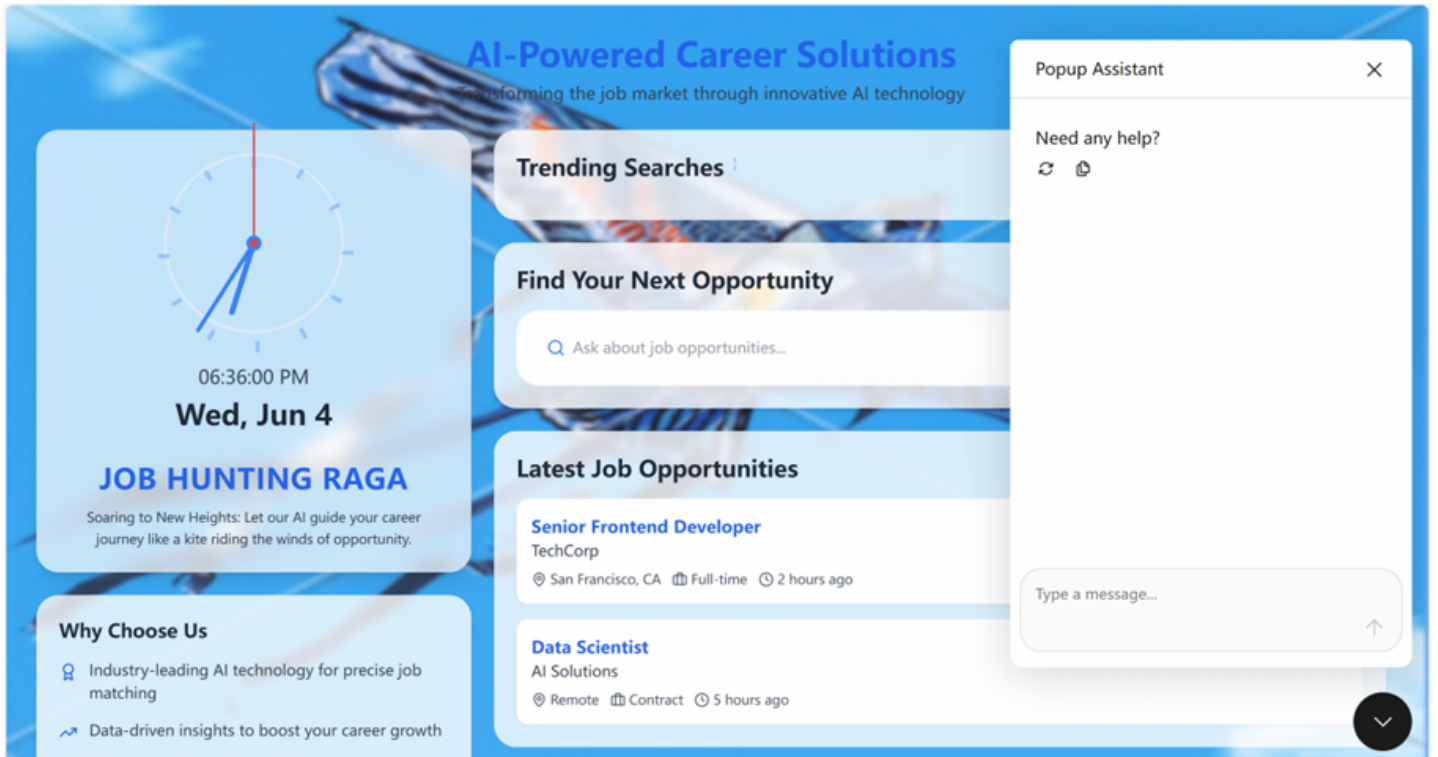
岗位智能推荐系统通过实时数据管道实现分钟级岗位信息同步，确保推荐内容的时效性。系统采用基于增量学习的推荐算法更新机制，持续优化推荐效果。推荐引擎综合考虑用户画像特征与岗位需求矩阵进行智能匹配，生成个性化推荐流，同时通过智能去重机制基于用户行为日志过滤重复曝光内容并控制推荐多样性，提升用户体验。为了便于用户能够获得更有效求职信息，我们设计了此项功能。开发时要注意该功能的时效性和准确性，同时要考虑页面美观性。



2.6 AI智能客服

功能描述:

AI客服功能为用户提供7×24小时的在线咨询服务。用户可以通过文字方式向AI客服提问，AI将快速准确地回答关于网站功能、使用方法、求职建议、岗位信息等相关问题。AI客服具备强大的知识库，能够实时更新和学习新的知识，确保回答的准确性和及时性。同时，AI客服支持多轮对话，能够根据用户的问题逐步深入解答，引导用户更好地了解网站的各项服务。开发时要注重AI客服的交互性和知识库的丰富性，让用户能够快速获得满意的解答。



2.7 职业测评与决策支持系统

功能描述：

职业测评与决策支持系统采用MBTI+大五人格模型的混合评估框架，从技术硬实力和软技能两个维度进行全面评估。系统根据测评结果自动生成包含竞争力指数等关键指标的个性化报告，并绘制清晰的职业发展路径图。推荐策略引擎采用基于强化学习的多目标优化算法，综合考虑薪资、成长性等因素，并实时融合市场供需指数，确保推荐结果的准确性和实用性。职业测评能够帮助用户了解自己的性格与能力，并明确其在职场中的作用，使用户能够找到更加适合自己的职业。同时，改进与建议部分能够让用户明确自己可以提升的方向，为用户未来的求职道路和个人成长提升大有裨益。

2.8 岗位精准搜索

功能描述：

精准工作搜索功能通过多维度的筛选条件，帮助用户快速定位到目标岗位。用户可以根据公司名称、岗位类型、工作地点等关键信息进行组合搜索，系统将实时匹配并展示符合条件的岗位列表。同时，搜索结果页面提供岗位详情的预览功能，用户可以快速浏览岗位职责、任职要求、公司介绍等核心信息，点击岗位名称即可进入详细页面，查看完整的岗位描述、福利待遇、面试流程等内容。开发时要注重搜索算法的精准度和响应速度，确保用户能够高效地找到心仪的工作。



2.9 简历生成

功能描述:

简历生成功能为用户提供便捷的简历制作服务。用户只需上传个人基本信息、教育背景、工作经历、项目经验、技能证书等资料，系统将基于预设的多种简历模板，一键生成专业、美观的简历文档。同时，用户可以查看简历的排版效果，确保简历的质量。此外，生成的简历支持PDF种格式导出，方便用户投递求职。开发时要注重模板的多样性和简历内容的逻辑性，让用户轻松制作出高质量的简历。

[← 返回首页](#)

专业简历生成器

填写信息，一键生成专业PDF简历

基本信息

姓名 *

请输入您的姓名

联系电话 *

请输入您的联系电话

个人照片

上传照片

个人简介 *

请简要介绍您的专业背景、技能特长和职业目标

教育经历

教育经历 1

2.10 AI智能面试

功能描述:

AI面试功能模拟真实面试场景，为用户提供沉浸式的面试体验。用户进入面试界面后，AI将作为面试官，根据岗位要求和用户简历信息，随机生成一系列面试问题，涵盖专业知识、沟通能力、团队协作、问题解决等多个维度。用户通过语音或文字回答问题，AI将实时对回答进行分析评估，从回答的完整性、逻辑性、准确性等方面给出评分，并在面试结束后生成详细的面试报告。报告中不仅包含面试问题和用户的回答内容，还会提供针对性的改进建议，帮助用户发现自身不足，提升面试技巧。开发时要注重AI对话的自然流畅性以及评分算法的科学性，确保面试结果的客观公正。



2.11 企业背景调查平台

功能描述:

企业背景调查平台为用户提供全面的企业信息查询服务。用户只需输入企业名称，系统将通过多渠道数据整合，快速展示企业的详细信息，包括但不限于企业基本信息、企业规模、经营范围等，帮助用户深入了解企业的背景和潜在风险。同时附有信息来源文章，确保信息真实性，也便于用户通过文章进一步了解企业。开发时要注重数据的准确性和更新及时性，确保用户能够获取到最新、最全面的企业信息。



3 AI技术细节

3.1 数据库向量检索

我们设计并实现了一套高效的检索机制，其核心为 `VectorStore` 类。该类深度整合了MySQL数据库与FAISS（Facebook AI Similarity Search）向量检索引擎，实现了高效的向量化相似度检索。并实现了对长文本的有效分割与处理

3.1.1 核心组件： `VectorStore` 类

`VectorStore` 类采用单例模式设计，确保在系统全局范围内维护唯一的数据库连接和FAISS索引实例，以优化资源管理和访问效率。

关键属性：

- `_instance`：静态属性，存储 `VectorStore` 类的单例对象。
- `_initialized`：布尔型标志，用于确保 `_init` 构造方法仅执行一次，防止重复初始化。
- `connection`：MySQL数据库的活动连接对象。
- `cursor`：数据库游标，用于执行SQL查询操作。
- `dimension`：文本嵌入向量的维度，系统默认为768。
- `job_hunting_index`：FAISS索引对象，专用于 `jobHunting` 数据表的向量化内容检索。
- `job_detail_index`：FAISS索引对象，专用于 `jobDetail` 数据表的向量化内容检索。
- `job_hunting_ids`：列表结构，存储与 `jobHunting` 表FAISS索引条目相对应的原始文档ID。
- `job_detail_ids`：列表结构，存储与 `jobDetail` 表FAISS索引条目相对应的原始文档ID。
- `config_path`：字符串，指向数据库配置文件的路径，默认为执行脚本同级目录下的 `config.json`。

- `db_config`: 字典类型, 存储从配置文件加载的数据库连接参数。

核心方法:

- `__new__()`: 类方法, 实现单例模式, 确保全局仅存在一个 `VectorStore` 实例。
- `__init__(config_path="config.json")`: 构造方法, 负责配置加载、数据库连接建立、FAISS索引的初始化与加载等关键步骤。
- `__load_config()`: 私有方法, 从指定的JSON配置文件中加载数据库连接参数。
- `__load_index()`: 私有方法, 尝试从本地持久化存储中加载预构建的FAISS索引及对应的ID映射表。若加载失败, 则触发索引重建流程。
- `__build_index()`: 私有方法, 核心的索引构建逻辑。该方法从MySQL数据库中提取文本内容的嵌入向量 (通常为BLOB类型), 将其转换为NumPy数组, 并分别构建 `job_hunting_index` 和 `job_detail_index` 这两个FAISS索引。构建完成后, 索引及ID映射将序列化并保存至本地, 供后续快速加载。
- `__convert_blob_to_numpy(blob_data)`: 私有辅助方法, 用于将从MySQL中检索到的BLOB格式向量数据转换为FAISS兼容的NumPy数组格式。
- `search(query_vector, top_k=5, mode=1)`: 公开接口, 执行纯向量相似度搜索。根据 `mode` 参数 (1代表 `jobHunting` 表, 其他值代表 `jobDetail` 表) 选择相应的FAISS索引, 基于余弦相似度返回最相似的 `top_k` 个文档ID及其相似度得分。
- `hybrid_search(query_vector, query_text, top_k=5, mode=1)`: 公开接口, 实现一种两阶段混合检索策略, 结合了向量语义相似度和关键词词法匹配的优势。
- `__chunk_text(text, chunk_size=500, overlap=100)`: 私有方法, 用于长文本分割。该方法以句子边界为优先分割点, 生成具有一定重叠 (`overlap`) 的文本块 (`chunks`), 以保证切分后文本片段的语义完整性。
- `__process_document_text(doc, mode)`: 私有辅助方法, 根据不同的模式 (`mode`) 将原始文档内容处理成适合进行文本匹配的字符串格式。
- `sel_job_hunting()`: 公开接口, 用于从 `jobHunting` 数据表读取数据并进行结构化处理。
- `sql_job_detail()`: 公开接口, 用于从 `jobDetail` 数据表读取数据并进行结构化处理。
- `__del__()`: 析构方法, 在对象销毁时确保数据库连接被正确关闭, 释放相关资源。

检索实现逻辑:

`VectorStore` 类构建了一个基于MySQL存储 + FAISS检索引擎的向量化检索系统, 并内建了混合搜索能力。系统为 `job_hunting` 和 `job_detail` 两张核心数据表分别构建了独立的FAISS索引。具体的检索流程如下:

1. 初步候选池构建 (向量检索阶段):

接收到用户查询 (`query`) 后, 首先将其转换为查询向量 (`query_vector`)。

系统根据 `mode` 参数选择目标FAISS索引 (`jobHunting` 或 `jobDetail`)。

利用FAISS执行高效的余弦相似度计算, 从选定索引中召回与查询向量语义最相近的 `top_k * 4` 个候选文档。此处的 `* 4` 是为了扩大初始候选池, 为后续的混合检索提供更丰富的筛选基数。

2. 精确排序与筛选 (混合检索阶段):

对初步候选池中的每个文档进行文本预处理。

针对长度超过500字符的长文档, 调用 `__chunk_text` 方法进行智能分块, 优先保证句子层面的语义完整性。

计算查询文本 (`query_text`) 中的关键词与处理后文档文本 (或其文本块) 之间的词法匹配得分 (如BM25或TF-IDF, 具体实现未在描述中详述, 但暗示了关键词匹配)。

综合向量相似度得分（权重70%）与关键词匹配得分（权重30%），计算得到一个混合排序分。

依据此综合分数对候选文档进行重新排序，最终筛选出 `top_k * 2` 个最优结果。此处的 `* 2` 旨在为生成优化阶段的重排序模块提供适量的、高质量的候选文档。

3.2 检索结果重排序（Re-ranking）

我们引入了一个更精细的重排序阶段，即对向量检索出来的 `top_k * 2` 个文档进行重排序，按照算法排序挑出其中的`top_k`个文档作为最终选定文档，为此，我们设计并实现了 `Retriever` 类，专门负责对 `VectorStore` 检索出的候选文档进行二次的、更深层次的重排序。

核心组件: `Retriever` 类

关键属性:

- `vectorDatabase`: 类型为 `VectorStore`，作为向量数据库的接口，用于获取初步检索的候选文档集。
- `model`: `SentenceTransformer` 类的实例，采用 `paraphrase-albert-small-v2` 预训练模型，用于在本地生成文本的语义嵌入向量。
- `tfidf`: `TfidfVectorizer` 类的实例，用于计算文本的TF-IDF（词频-逆文档频率）特征，支持关键词匹配与评分机制。
- `stop_words`: 集合类型，包含了从NLTK（Natural Language Toolkit）库获取的英文停用词列表，用于文本预处理以提升TF-IDF的准确性。
- `embeddings`: `OpenAIEmbeddings` 类的实例，通过调用OpenAI API生成高维（768维）文本嵌入向量，提供高质量的语义表征。

核心方法:

- `__init__(vectorDatabase: VectorStore)`: 构造方法，负责初始化各类嵌入模型（`SentenceTransformer`，`OpenAIEmbeddings`）、TF-IDF向量化器、NLTK相关资源，并关联传入的 `VectorStore` 实例。
- `_normalize_text(text: str) -> str`: 私有文本预处理方法，移除输入文本中的标点符号并将所有字符转换为小写，以确保后续处理的一致性与准确性。
- `_chunk_text(text: str, chunk_size=500, overlap=100) -> List[str]`: 私有长文本分割方法，将长文本切分为具有重叠区域的文本块，适用于大规模文本处理及计算平均嵌入向量的场景。
- `_mmr_diversity(query_embedding, doc_embeddings, lambda_param=0.7) -> List[int]`: 私有方法，实现Maximal Marginal Relevance (MMR)算法。该算法旨在对检索结果进行多样性重排序，通过 `lambda_param` 参数（默认为0.7，偏向相关性）平衡文档与查询的相关性以及已选文档间的差异性，以减少信息冗余。
- `_hybrid_search(query, documents, top_k=5, similarity_threshold=0.3) -> List[Tuple[float, Dict]]`: 私有方法，实现一个定制化的混合搜索与重排序逻辑。它结合了由 `SentenceTransformer` 计算的语义相似度和TF-IDF计算的词法相似度，对输入文档集进行加权评分、基于阈值（`similarity_threshold`）的过滤以及最终排序。

- `retrive_data(query, top_k=5, mode=1, similarity_threshold=0.3) -> List[Dict[str, Any]]`：公开的核心检索与重排序接口。该方法首先调用 `vectorDatabase`（即 `VectorStore` 实例）的 `hybrid_search` 获取初步候选集，随后应用本类的 `hybrid_search` 逻辑进行精细排序，并可选择性地整合MMR算法提升结果多样性，最终格式化输出结果（支持不同搜索模式 `mode`）。
- `generate_content_embedding(text: str) -> List[float]`：公开方法，用于为输入文本生成嵌入向量。对于长文本，该方法会先进行分块处理，然后计算各分块嵌入向量的平均值作为整体文本的表征，支持本地嵌入向量的提取流程。

重排序实现逻辑：

`Retriever` 类对 `VectorStore` 混合检索阶段输出的 `top_k * 2` 个候选文档执行以下更精细的重排序流程：

1. 混合相似度计算：

- **语义相似度**：利用 `SentenceTransformer` 模型（`paraphrase-albert-small-v2`）计算查询与各候选文档之间的语义相似度得分。
- **词法相似度**：利用TF-IDF模型计算查询与各候选文档基于关键词的文本相似度得分。
- 将语义相似度得分（权重70%）与TF-IDF得分（权重30%）进行加权融合，得到综合相似度评分。

2. 多样性增强（MMR算法）：

在获得综合相似度评分的文档列表基础上，应用MMR（Maximal Marginal Relevance）算法。该算法通过 `lambda` 参数（设定为0.7）来平衡结果的相关性与多样性，旨在避免最终结果中出现内容高度相似的多个文档，提升信息丰富度。

3. 阈值过滤与最终选择：

根据MMR算法重排序后的结果，筛选出综合相似度得分高于预设阈值（0.3）的文档。

从满足条件的文档中，选取最终的 `top_k` 个作为重排序阶段的输出，供后续生成模型使用。

3.3 智能化网络搜索与内容提炼模块优化

在先进的检索增强生成（RAG）应用中，整合实时网络搜索能力作为外部知识源的动态补充，对于提升系统响应的即时性与信息覆盖广度具有不可替代的战略价值。然而，本系统在该模块的早期实现（原 `websearch` 组件）面临以下严峻挑战：

- **原始网页信息获取的局限性**：早期采用基于标准库（如 `requests`）的直接网页爬取策略，在实践中频繁遭遇目标网站反爬虫机制、客户端动态渲染内容（JavaScript执行依赖）等技术壁垒，导致信息获取失败率高或关键内容遗漏，严重制约了数据源的可靠性。
- **低信噪比的原始网页内容**：即便成功获取网页数据，其原始内容往往夹杂大量与用户查询意图无关的元素，例如广告区块、导航链接、重复页眉页脚以及混乱的HTML结构。此类未经处理的“噪声”数据若直接注入RAG流程，极易误导大语言模型（LLM）的语义理解与信息筛选，从而显著降低最终生成答案的准确性与相关性。

为系统性解决上述瓶颈，我们对网络搜索模块进行了架构性升级与智能化改造，核心举措包括引入第三方专业级AI搜索服务，并串联自主研发的智能内容深度处理 workflow：

1. 集成“博查AI”专业级Web搜索服务作为稳定数据源：

- 系统摒弃了传统的直接网页抓取方式，转而采用集成“博查AI”（Bocha AI）提供的专业Web搜索API。这一转换通过新构建的 `WebSearch` 类进行封装与统一管理，该类作为与外部搜索服务交互的标准化接口。

- 其核心功能由异步方法 `query(search_terms)` 承担，负责执行以下搜索流程：
 - 查询构建**：接收外部传入的搜索关键词集合（`search_terms`），并将其聚合成符合API要求的查询字符串。
 - 请求参数化**：精心构造POST请求体，其中包含对搜索行为进行精确控制的关键参数，例如：用户查询字符串（`query`）、期望的内容新鲜度（`freshness`，配置为 `noLimit` 以获取最广泛时间范围的结果）、是否启用直接答案模式（`answer`，配置为 `True` 以尝试获取更直接的结果）、以及单次请求返回的最大结果数量（`count`，本期配置为3条，以平衡信息量与处理效率）等。
 - API调用与结果获取**：通过安全的HTTPS POST请求与博查AI的搜索服务进行通信，并接收返回的结构化JSON数据。此数据通常包含多个搜索条目，每个条目附带URL、标题、文本片段等元信息。

2. 构建基于 `ExtractKeyPhase` 的智能内容摘要与关键信息精炼管道：

- 针对博查AI返回的初步搜索结果（本质上仍是浓缩的网页片段，可能包含冗余或非核心信息），我们设计并集成了 `ExtractKeyPhase` 组件，用于进行二次的、面向LLM的深度内容处理。
- 在 `WebSearch` 类的实现中，内部实例化了 `ExtractKeyPhase`，并将其作为专用的内容摘要与提炼引擎（`summaryEngine`）。
- 当 `WebSearch` 从博查AI获取原始搜索数据（`content`）后，会立即调用 `summaryEngine.summary(search_term, content)` 异步方法，启动内容精炼流程。
- `ExtractKeyPhase` 组件的 `summary` 方法（其内部进一步调用 `generate_summary` 方法）是此阶段的核心。它依据预设的、针对性的提示词模板（`SummaryPrompt`），精确引导大语言模型（LLM）执行以下认知任务：
 - 上下文感知理解**：基于用户最初的查询意图（`search_term`），对输入的网络原始内容（`webcontent`）进行深度语义分析与上下文情景理解。
 - 关键信息识别与抽取**：从文本中精准识别并抽取与查询意图直接相关的核心事实、关键论点或重要数据。
 - 高度凝练的摘要生成**：将抽取到的关键信息进行重组与概括，生成一段语言精炼、信息密度高、且与用户查询紧密关联的文本摘要（`summaryContent`）。
- 通过这一“专业搜索 + 智能精炼”的两阶段处理流程，确保了最终注入RAG系统核心生成模块的外部网络信息，是经过严格筛选、深度加工和高度浓缩的高质量知识片段。此举从源头上显著降低了LLM受到互联网无关信息干扰的风险，提升了外部知识的可用性和有效性。

3.4 raga检索链构建

结合上面的一些关键技术，就可以构建出完整的raga搜索链，主要有以下步骤：

用户意图识别

使用意图识别模型或 LLM 进行输入分析，判断用户查询类型（如找职位、问福利、简历建议），根据判断出来的意图来选择合适的数据库进行查询降低对于多数据库同时查询的时延。

多源文档检索

提取出原始查询，将查询转为向量，作为索引检索的输入。使用 FAISS 向量数据库对意图识别得到的相关数据库进行检索，得到 $\text{top_k} * 2$ 个最相关文档。

文档选择与上下文构建

对检索结果进行重排，得到 top_k 个最相关文档，并进行阈值判断，如果相似度得分不高于预设阈值（0.3），则不采纳，防止对回答产生不良影响。并与 websearch 得到的数据进行结合，得到 raga 搜索的上下文。

知识增强生成

将上下文传入 LLM 中，基于我们给定的 prompt 模板进行回答生成。支持结构化输出（JSON 格式职位推荐、技能分析）与自然语言回答并存。

多轮控制与状态管理

记录用户历史查询与系统输出，用于后续问题上下文记忆（例如用户已询问过“岗位地点”）。我们的系统可根据用户反馈（如“不喜欢这个岗位”）自动调整下次检索策略。